МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ

по лабораторной работе №1
по дисциплине «Основы нейронных сетей»
Тема: СВЁРТОЧНЫЕ СЕТИ

Студент гр. 3312	_____	Барченков П.А.
Студент гр. 3312	_____	Лебедев И.А.
Студент гр. 3312	_____	Шарапов И.Д.
Преподаватель	_____	Петруша П.Г.

Санкт-Петербург
2026

Содержание

ЦЕЛЬ РАБОТЫ 3

ОПИСАНИЕ ЗАДАНИЯ 3

ПОДГОТОВКА ДАННЫХ 3

СТРУКТУРА МОДЕЛЕЙ 4

ОБУЧЕНИЕ МОДЕЛЕЙ 5

ВЗАИМОДЕЙСТВИЕ С МОДЕЛЯМИ 7

ВЫВОДЫ 9

ПРИЛОЖЕНИЕ 10

ЦЕЛЬ РАБОТЫ

Построение и изучение свёрточных нейронных сетей для классификации изображений.

ОПИСАНИЕ ЗАДАНИЯ

Самостоятельно выбрать датасет на сайте kaggle.com.

Разделить данные на тестовую, тренировочную и валидационную выборки.

Разработать собственную архитектуру свёрточной сети. Протестировать её вместе с готовыми моделями: AlexNet, VGG16, VGG19, ResNet.

Проанализировать основные метрики.

ПОДГОТОВКА ДАННЫХ

В качестве датасета был выбран *«Bird vs Drone»* https://www.kaggle.com/datasets/truong9/bird-vs-drone. Датасет содержит более 20 000 изображений двух классов: «птица» и «дрон». Соотношение количества изображений класса примерно 2 к 3.

Сначала вручную все изображения из оригинального датасета были перемещены в одну папку *dataset*. Для валидационной выборки отобрано по 10 изображений каждого класса и помещено в подпапки *valid/bird* и *valid/drone*.

С помощью функции `get_data_generators()` создаются переменные `train_gen`, `valid_gen`, `test_gen`, которые можно подавать моделям на вход. Для этого внутри функции данные делятся в соотношении 20 к 80, в зависимости от типа модели задаются необходимые параметры, фотографии приводятся к одинаковому размеру и производится аугментация (случайный поворот, смещение и т.п.). В конце с помощью `flow_from_dataframe()` создаются и возвращаются объекты класса `DataFrameIterator`.

СТРУКТУРА МОДЕЛЕЙ

Все модели реализованы в модуле main.py с помощью tensorflow.keras.

Собственная модель MyCNN:

```
model = models.Sequential(name='MyCNN')
model.add(layers.Input(shape=input_shape))
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(num_classes))
```

Простая свёрточная сеть из трёх блоков `Conv2D` + `MaxPooling2D`. После извлечения признаков применяется полносвязный классификатор с Dropout.

Готовые модели:

AlexNet – классическая архитектура 2012 года, реализованная вручную: `Conv(11×11, 96) → MaxPool → Conv(5×5, 256) → MaxPool → три Conv(3×3, 384/384/256) → MaxPool → два Dense(4096) с Dropout(0.3) → Dense(2)`.

VGG16 и VGG19 – используют предобученные backbone (`weights='imagenet', include_top=False`). Backbone заморожен (`trainable=False`). Добавлены: `GlobalAveragePooling2D → Dense(512, relu) → Dropout(0.5) → Dense(2)`. Применяется transfer learning.

ResNet – собственная реализация ResNet-20 (n=3): Начальный `Conv` + 3 группы `residual blocks` с `BatchNormalization`, `ReLU` и `skip-connections` (`HeNormal` инициализация). Завершается `GlobalAveragePooling2D + Dense(2)`. Обучается с нуля.

Общие параметры обучения: Optimizer: `Adam`, Loss: `SparseCategoricalCrossentropy(from_logits=True)`, Metric: `accuracy`.

Количество эпох задаётся пользователем (в экспериментах 20)

Модели сохраняются в папку *models/* (формат .keras), метрики — в *metrics/*.

ОБУЧЕНИЕ МОДЕЛЕЙ

Обучение проведено для всех пяти моделей (20 эпох). Для каждой модели автоматически сохраняются:

- файл *_metrics.txt* (имя модели, эпохи, test loss/accuracy, train accuracy, val accuracy);
- график *_training.png* (кривые accuracy и loss для train/val).

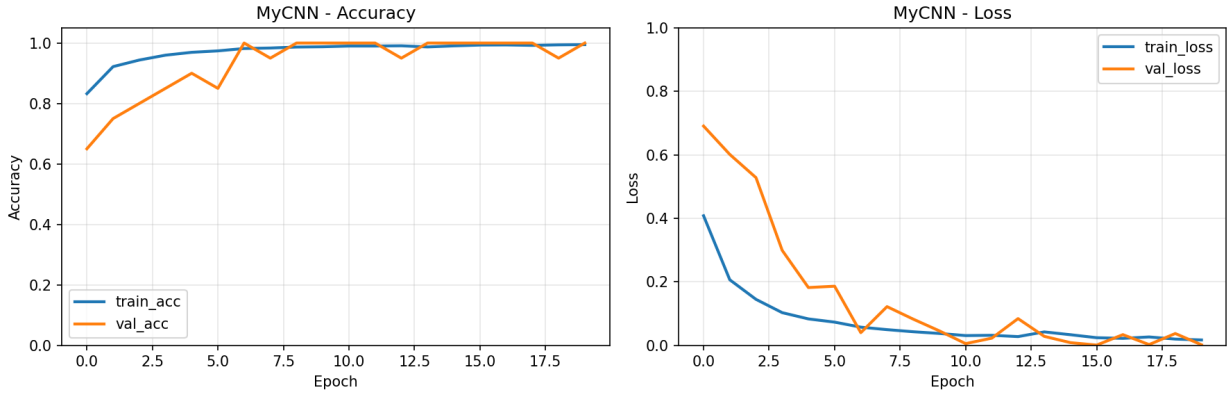

Рисунок 1 – Анализ модели MyCNN

Accuracy MyCNN быстро сходится к 1, а loss стремится к нулю. На тестовой выборке получили значения loss: 0.0171, accuracy: 0.9945. Обучение в среднем: 170мс/шаг.

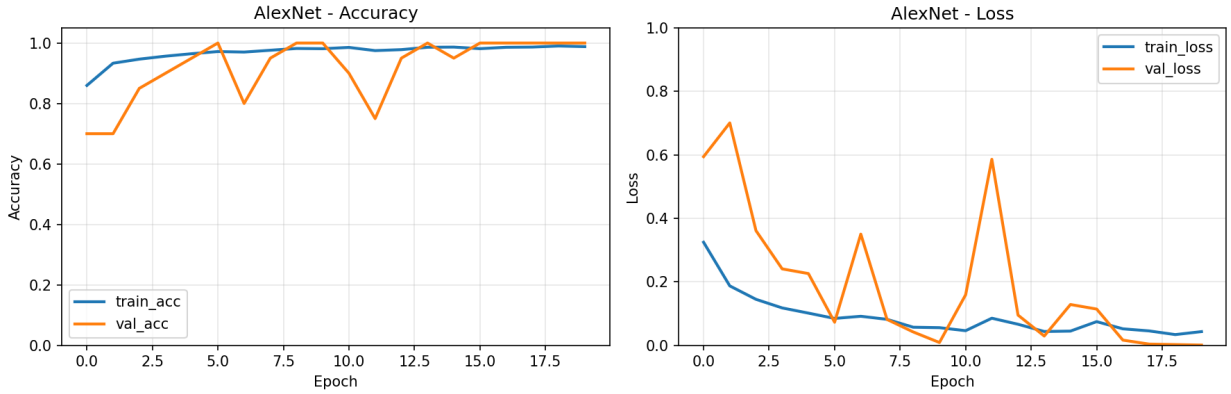

Рисунок 2 – Анализ модели AlexNet

Accuracy AlexNet также быстро сходится к 1, а loss стремится к нулю, но подходит не так близко, как у MyCNN. На тестовой выборке получили значения loss: 0.0082, accuracy: 0.9967. Обучение в среднем: 180мс/шаг.

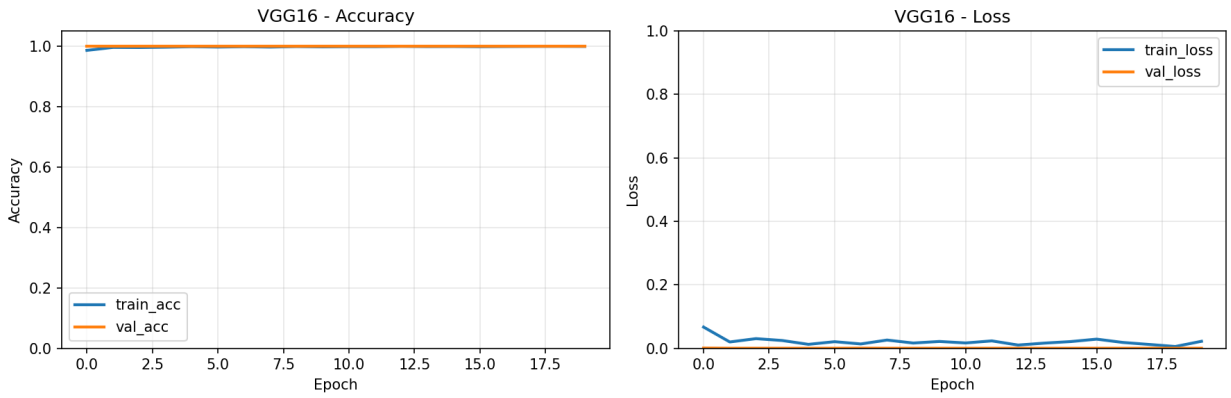

Рисунок 3 – Анализ модели VGG16

Accuracy VGG16 с первой же эпохи приближается к 1, а loss к нулю. На тестовой выборке получили значения loss: 0.0247, accuracy: 0.9993. Обучение в среднем: 160мс/шаг.

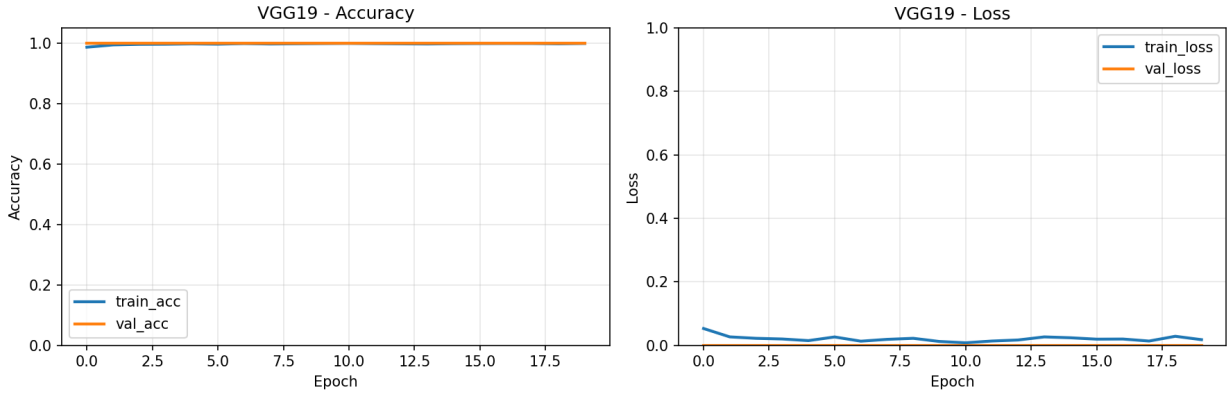

Рисунок 4 – Анализ модели VGG19

Ситуация с VGG19 аналогична VGG16. На тестовой выборке получили значения loss: 0.0108, accuracy: 0.9993. Обучение в среднем: 190мс/шаг.

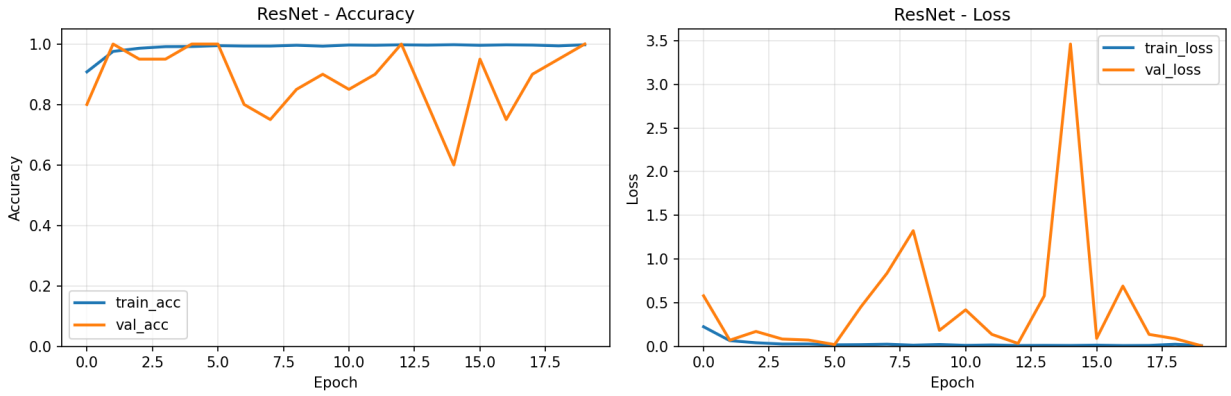

Рисунок 5 – Анализ модели ResNet

Характер графиков ResNet схож с моделями VGG16 и VGG19. На тестовой выборке получили значения loss: 0.0204, accuracy: 0.9935. Обучение в среднем: 210мс/шаг.

ВЗАИМОДЕЙСТВИЕ С МОДЕЛЯМИ

Для обучения и использования моделей было написано консольное приложение с меню (рис. 6).

```
========================================
              ГЛАВНОЕ МЕНЮ
========================================
1. Обучение модели
2. Определить предмет на картинке
0. Выйти
Выберите пункт:
```

Рисунок 6 – Меню приложения

В «Обучение модели» предлагается выбрать тип модели и количество эпох (рис. 7). Полученные модели, сохраняются в папку *models*, а значения accuracy, loss и графики в папку *metrics*. Все файлы в названии имеют временную метку, чтобы отличать модели одного и того же типа, обученные в разное время и на разном количестве эпох.

```
========================================
       ВЫБОР МОДЕЛИ ДЛЯ ОБУЧЕНИЯ
========================================
1. MyCNN
2. AlexNet
3. VGG16
4. VGG19
5. ResNet
0. Назад
Выберите модель:
```

Рисунок 7 – Выбор модели для обучения

При выборе «Определить предмет на картинке» выпадает список обученных моделей (из папки *models*) (рис. 8).

```
========================================
            ДОСТУПНЫЕ МОДЕЛИ
========================================
1. AlexNet_20260318_172104.keras
2. AlexNet_20260320_113926.keras
3. AlexNet_20260329_160144.keras
4. AlexNet_20260329_183159.keras
5. MyCNN_20260318_164916.keras
6. ResNet_20260319_002754.keras
7. VGG16_20260318_175010.keras
8. VGG19_20260318_182309.keras
0. Назад
Выберите номер модели:
```

Рисунок 8 – Список обученных моделей

После выбора модели приложении предлагает вывести список поддерживаемых файлов в корневой папке (рис. 9). Ждёт на вход путь до файла и выводит предположение того, что на картинке и выводит вероятности отношения к классам.

```
========================================
            АНАЛИЗ ИЗОБРАЖЕНИЯ
========================================
Модель: VGG16
Epochs: 20
Train accuracy: 0.9987 | Val accuracy: 1.0000
Test loss: 0.0247 | Test accuracy: 0.9993
(Для выхода в главное меню введите "0" вместо пути)
Показать список изображений в текущей папке? (y/n): y
Найденные изображения в текущей папке:
  bird.jpg
  drone.jpg

Введите путь к изображению: bird.jpg
1/1 [==============================] - 0s 90ms/step
Результат: bird
Вероятности: [1. 0.]

Введите путь к изображению: drone.jpg
1/1 [==============================] - 0s 17ms/step
Результат: drone
Вероятности: [0. 1.]

Введите путь к изображению:
```

Рисунок 9 – Пример анализа изображения на VGG16

ВЫВОДЫ

В ходе лабораторной работы были построены и исследованы свёрточные нейронные сети для задачи классификации изображений на датасете *«Bird vs Drone»*. Были подготовлены обучающая, валидационная и тестовая выборки, а также реализована собственная архитектура MyCNN и протестированы готовые модели AlexNet, VGG16, VGG19 и ResNet.

Полученные результаты показали, что все рассмотренные модели успешно справляются с задачей классификации, демонстрируя высокие значения accuracy на тестовой выборке. Наилучшие показатели качества показали VGG16 и VGG19, а собственная модель MyCNN и ResNet также продемонстрировали хорошие результаты при меньшей вычислительной сложности.

В ходе работы было установлено, что использование transfer learning для моделей VGG16 и VGG19 позволяет добиться очень высокой точности даже при сравнительно небольшом числе обучающих эпох.

Свёрточные нейронные сети являются эффективным инструментом, а выбор архитектуры влияет как на точность, так и на скорость обучения.

ПРИЛОЖЕНИЕ

Ссылка на GitHub: https://github.com/DexTver/NN_LETI/tree/lab01